Rappresentazione dei numeri

Indice

Introduzione

Rappresentazione dei numeri naturali

Rappresentazione dei numeri interi

Rappresentazione dei numeri reali

In sintesi

Introduzione

Gli algoritmi manipolano dati

- Un algoritmo manipola dati: prima di descrivere come funziona, dobbiamo stabilire come questi dati vengono **scritti**
- Per i numeri la risposta è precisa: un calcolatore memorizza ogni numero come una **sequenza finita di simboli**
- Con un dato numero di simboli si possono scrivere soltanto **un numero finito** di numeri

Due conseguenze importanti

- Ogni rappresentazione ha un **intervallo di rappresentazione**: i numeri al di fuori non possono essere memorizzati
- La rappresentazione dei numeri reali è, in generale, solo **approssimata**

Tre rappresentazioni

In questo capitolo studiamo:

- la rappresentazione dei **numeri naturali** in un sistema di numerazione posizionale in base b, in particolare il sistema binario usato dai calcolatori;
- la rappresentazione dei **numeri interi**, prima in modulo e segno e poi in complemento a due, la rappresentazione usata dai calcolatori moderni;
- la rappresentazione dei **numeri reali** in virgola mobile, secondo lo standard IEEE 754.

Per ogni rappresentazione calcoliamo il numero più grande e quello più piccolo che si possono rappresentare con un dato numero di cifre, e il numero minimo di cifre necessarie per rappresentare un dato numero.

Rappresentazione dei numeri naturali

Introduzione: Rappresentazione dei numeri naturali

- Definiamo i sistemi di numerazione posizionali e la loro base
- Calcoliamo il numero rappresentato da una sequenza di cifre in base b
- Troviamo quali numeri naturali si scrivono con k cifre, e quante cifre servono per un dato numero
- Il sistema binario è quello usato dai calcolatori

Sistemi di numerazione posizionali

- Siamo abituati a scrivere i numeri nel sistema decimale
- La stessa cifra ha un valore diverso a seconda della posizione in cui compare
- In 555 le tre cifre sono uguali, ma valgono 500, 50 e 5
- Questa è l'idea di **sistema di numerazione posizionale**

Definizione 1: sistema di numerazione posizionale

*Un **sistema di numerazione posizionale** è un modo di codificare i numeri naturali mediante una **sequenza di cifre**, in cui ogni cifra (singolo simbolo) contribuisce in modo diverso a seconda della sua posizione.*

Definizione 2: base di un sistema di numerazione posizionale

*Il numero di cifre diverse di un sistema di numerazione posizionale si chiama **base** b del sistema.*

Basi

- La base b di un sistema di numerazione posizionale deve essere maggiore o uguale a due
- Il **sistema unario** non è un sistema di numerazione posizionale: i numeri naturali si rappresentano ripetendo la cifra 1 tante volte quanto necessario
  - ad esempio, 3 si rappresenta come 111

I principali sistemi di numerazione posizionali

Sistema di numerazione posizionale	base b	cifre diverse
Sistema binario	2	0, 1
Sistema ottale	8	0, 1, 2, 3, 4, 5, 6, 7
Sistema decimale	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Sistema esadecimale	16	$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F$

- Nel sistema esadecimale le sei lettere A, B, C, D, E, F sono cifre, con i valori 10, 11, 12, 13, 14, 15

Definizione 3: sequenza di cifre

*Una **sequenza di** k **cifre** che rappresenta un numero naturale in base b si indica con:*

$$\left(d_{k-1} \quad d_{k-2} \quad \ldots \quad d_2 \quad d_1 \quad d_0\right)_b$$

*dove le posizioni si contano da destra a sinistra, partendo da 0 e crescendo fino a $k-1$. La **cifra meno significativa** (LSD, Least Significant Digit) è d_0. La **cifra più significativa** (MSD, Most Significant Digit) è d_{k-1}.*

Il valore di una sequenza di cifre

- La cifra in posizione i viene moltiplicata per la potenza i-esima della base
- Sommando tutti questi contributi si ottiene il numero rappresentato dalla sequenza

Data una sequenza di k cifre in un sistema di numerazione posizionale in base b, il numero naturale che essa rappresenta in base dieci è dato dalla seguente sequenza di cifre:

$$\left(\sum_{i=0}^{k-1} d_i \; b^i \right)_{10}$$

Esempio: sequenze di cifre (1/2)

- Il numero cento scritto nei quattro sistemi di numerazione posizionali della tabella precedente
- Il valore di ogni sequenza si calcola con la formula $\sum_{i=0}^{k-1} d_i \, b^i$, posizione per posizione, partendo dalla cifra meno significativa d_0
- Guardiamo prima il sistema binario, poi gli altri tre

Esempio: sequenze di cifre (2/2)

Sequenza	b	k	rappresentazione decimale
$(100)_2$	2	3	$(0 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2)_{10} = (4)_{10}$
$(1100100)_2$	2	7	$(0 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 0 \cdot 2^4 + 1 \cdot 2^5 + 1 \cdot 2^6)_{10}$ $= (4 + 32 + 64)_{10} = (100)_{10}$
$(144)_8$	8	3	$(4 \cdot 8^0 + 4 \cdot 8^1 + 1 \cdot 8^2)_{10} = (4 + 32 + 64)_{10} = (100)_{10}$
$(100)_{10}$	10	3	$(0 \cdot 10^0 + 0 \cdot 10^1 + 1 \cdot 10^2)_{10} = (100)_{10}$
$(64)_{16}$	16	2	$(4 \cdot 16^0 + 6 \cdot 16^1)_{10} = (4 + 96)_{10} = (100)_{10}$

La stessa sequenza di cifre 100 rappresenta numeri diversi in basi diverse, e lo stesso numero richiede più cifre quando la base è più piccola.

Con $k = 4$ cifre binarie possiamo scrivere $2^4 = 16$ sequenze diverse.

Vediamole tutte.

Esempio: tutte le sequenze di quattro cifre binarie (1/2)

Le 16 sequenze rappresentano i numeri naturali da 0 a 15:

Sequenza	b	k	rappresentazione decimale
$(0000)_2$	2	4	$(0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0)_{10} = (0)_{10}$
$(0001)_2$	2	4	$(0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0)_{10} = (1)_{10}$
$(0010)_2$	2	4	$(0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)_{10} = (2)_{10}$
$(0011)_2$	2	4	$(0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)_{10} = (3)_{10}$
$(0100)_2$	2	4	$(0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0)_{10} = (4)_{10}$
$(0101)_2$	2	4	$(0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0)_{10} = (5)_{10}$
$(0110)_2$	2	4	$(0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)_{10} = (6)_{10}$
$(0111)_2$	2	4	$(0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)_{10} = (7)_{10}$

Esempio: tutte le sequenze di quattro cifre binarie (2/2)

Sequenza	b	k	rappresentazione decimale
$(1000)_2$	2	4	$(1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0)_{10} = (8)_{10}$
$(1001)_2$	2	4	$(1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0)_{10} = (9)_{10}$
$(1010)_2$	2	4	$(1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)_{10} = (10)_{10}$
$(1011)_2$	2	4	$(1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)_{10} = (11)_{10}$
$(1100)_2$	2	4	$(1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0)_{10} = (12)_{10}$
$(1101)_2$	2	4	$(1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0)_{10} = (13)_{10}$
$(1110)_2$	2	4	$(1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)_{10} = (14)_{10}$
$(1111)_2$	2	4	$(1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)_{10} = (15)_{10}$

Notazione

Di solito, per le sequenze di cifre in base dieci,
le parentesi si omettono

e, in assenza di parentesi,
la base si assume uguale a dieci.

Intervalli di rappresentazione

- La tabella di tutte le sequenze di quattro cifre binarie si ferma a 15: con quattro cifre binarie non c'è modo di scrivere 16
- In generale, un numero fissato k di cifre può rappresentare solo i numeri naturali fino a un certo limite
- Calcoliamo ora questo limite

Osservazione 1

Date k cifre in base b, il ***più grande numero naturale*** *rappresentabile è:*

$$b^k - 1$$

e il relativo ***intervallo di rappresentazione*** *per i numeri naturali è:*

$$\left[\ 0,\ b^k - 1\ \right]$$

Dimostrazione (1/2)

Il più grande numero naturale in base b con k cifre è formato soltanto dalla cifra più grande $(b-1)$ ripetuta k volte:

$$\left((b-1)_{k-1} \;\; (b-1)_{k-2} \;\; \ldots \;\; (b-1)_1 \;\; (b-1)_0 \right)_b$$

Usando la somma della progressione geometrica (valida poiché $b \geq 2$)

$$\sum_{i=0}^{k-1} b^i = \frac{b^k - 1}{b-1}$$

il più grande numero naturale in decimale è:

$$\sum_{i=0}^{k-1} (b-1)\, b^i = (b-1) \sum_{i=0}^{k-1} b^i = (b-1)\, \frac{b^k - 1}{b-1} = b^k - 1$$

Dimostrazione (2/2)

- Il più piccolo numero naturale è 0, rappresentato da k cifre uguali a 0
- Ogni numero naturale compreso tra 0 e $b^k - 1$ può essere rappresentato:
  - le b^k sequenze diverse di k cifre rappresentano b^k numeri diversi
  - tutti appartengono all'intervallo $[0, b^k - 1]$
  - l'intervallo contiene esattamente b^k numeri naturali



Bit e byte

Le cifre del sistema binario si chiamano **bit**

e 8 bit formano un **byte**.

Esempio: intervalli di rappresentazione in binario

Per $b = 2$ l'intervallo di rappresentazione con k bit è $[0, 2^k - 1]$. Le ultime righe corrispondono a 1, 2, 4 e 8 byte.

k	b	$b^k - 1$	intervallo di rappresentazione
1	2	$2^1 - 1 = 1$	$[0, 1]$
2	2	$2^2 - 1 = 3$	$[0, 3]$
3	2	$2^3 - 1 = 7$	$[0, 7]$
4	2	$2^4 - 1 = 15$	$[0, 15]$
8	2	$2^8 - 1 = 255$	$[0, 255]$
16	2	$2^{16} - 1 = 65{,}535$	$[0, 65{,}535]$
32	2	$2^{32} - 1 = 4{,}294{,}967{,}295$	$[0, 4{,}294{,}967{,}295]$[1]
64	2	$2^{64} - 1$	$[0, 2^{64} - 1]$[2]

[1] Intervallo di rappresentazione di `unsigned int` in C e C++ (4 byte).

[2] Intervallo di rappresentazione di `unsigned long long int` in C e C++ (8 byte);
$2^{64} = 18{,}446{,}744{,}073{,}709{,}551{,}616$.

L'osservazione si può leggere anche nel verso opposto:

dato un numero naturale n,
quante cifre servono per scriverlo?

Osservazione 2

Il numero minimo di cifre per scrivere un numero naturale $n \geq 1$ *in base* b *è:*

$$\lfloor \log_b n \rfloor + 1$$

Dimostrazione

Per l'Osservazione 1, il numero n si può scrivere con k cifre se e solo se:

$$n \leq b^k - 1 \quad \Longleftrightarrow \quad n < b^k \quad \Longleftrightarrow \quad \log_b n < k$$

- la prima equivalenza vale perché n e b^k sono interi
- la seconda vale perché il logaritmo in base $b \geq 2$ è una funzione crescente

Il più piccolo intero k strettamente maggiore di $\log_b n$ è $\lfloor \log_b n \rfloor + 1$.

Di conseguenza, $\lfloor \log_b n \rfloor + 1$ è il numero minimo di cifre per scrivere un numero naturale n in base b. □

Esempio: numero minimo di cifre di cento

Calcoliamo il numero minimo di cifre di $n = 100$ in base 2 e in base 10.

- Passo 1 (base 2): poiché $2^6 = 64 \leq 100 < 128 = 2^7$, si ha $6 \leq \log_2 100 < 7$ (infatti $\log_2 100 \approx 6.644$), quindi:

$$\lfloor \log_2 100 \rfloor + 1 = 6 + 1 = 7$$

- Passo 2 (base 10): poiché $10^2 = 100$, si ha $\log_{10} 100 = 2$, quindi:

$$\lfloor \log_{10} 100 \rfloor + 1 = 2 + 1 = 3$$

Entrambi i risultati concordano con l'esempio sul numero cento: $(1100100)_2$ ha 7 cifre e $(100)_{10}$ ha 3 cifre.

Esempio: numero minimo di cifre in binario (1/2)

Per le potenze di due $n = 2^m$ si ha $\log_2 n = m$, quindi il numero minimo di bit è $m + 1$: una cifra 1 seguita da m cifre 0.

n	b	$\log_b n$	$\lfloor \log_b n \rfloor + 1$
$2 = 2^1$	2	$\log_2 2 = 1$	$\lfloor \log_2 2 \rfloor + 1 = 2$
$4 = 2^2$	2	$\log_2 4 = 2$	$\lfloor \log_2 4 \rfloor + 1 = 3$
$8 = 2^3$	2	$\log_2 8 = 3$	$\lfloor \log_2 8 \rfloor + 1 = 4$
$16 = 2^4$	2	$\log_2 16 = 4$	$\lfloor \log_2 16 \rfloor + 1 = 5$
$32 = 2^5$	2	$\log_2 32 = 5$	$\lfloor \log_2 32 \rfloor + 1 = 6$
$64 = 2^6$	2	$\log_2 64 = 6$	$\lfloor \log_2 64 \rfloor + 1 = 7$
$128 = 2^7$	2	$\log_2 128 = 7$	$\lfloor \log_2 128 \rfloor + 1 = 8$
$256 = 2^8$	2	$\log_2 256 = 8$	$\lfloor \log_2 256 \rfloor + 1 = 9$
$512 = 2^9$	2	$\log_2 512 = 9$	$\lfloor \log_2 512 \rfloor + 1 = 10$
$1{,}024 = 2^{10}$	2	$\log_2 1{,}024 = 10$	$\lfloor \log_2 1{,}024 \rfloor + 1 = 11$

Esempio: numero minimo di cifre in binario (2/2)

n	b	$\log_b n$	$\lfloor \log_b n \rfloor + 1$
$2{,}048 = 2^{11}$	2	$\log_2 2{,}048 = 11$	$\lfloor \log_2 2{,}048 \rfloor + 1 = 12$
$4{,}096 = 2^{12}$	2	$\log_2 4{,}096 = 12$	$\lfloor \log_2 4{,}096 \rfloor + 1 = 13$
$8{,}192 = 2^{13}$	2	$\log_2 8{,}192 = 13$	$\lfloor \log_2 8{,}192 \rfloor + 1 = 14$
$16{,}384 = 2^{14}$	2	$\log_2 16{,}384 = 14$	$\lfloor \log_2 16{,}384 \rfloor + 1 = 15$
$32{,}768 = 2^{15}$	2	$\log_2 32{,}768 = 15$	$\lfloor \log_2 32{,}768 \rfloor + 1 = 16$
$65{,}536 = 2^{16}$	2	$\log_2 65{,}536 = 16$	$\lfloor \log_2 65{,}536 \rfloor + 1 = 17$
$131{,}072 = 2^{17}$	2	$\log_2 131{,}072 = 17$	$\lfloor \log_2 131{,}072 \rfloor + 1 = 18$
$262{,}144 = 2^{18}$	2	$\log_2 262{,}144 = 18$	$\lfloor \log_2 262{,}144 \rfloor + 1 = 19$
$524{,}288 = 2^{19}$	2	$\log_2 524{,}288 = 19$	$\lfloor \log_2 524{,}288 \rfloor + 1 = 20$
$1{,}048{,}576 = 2^{20}$	2	$\log_2 1{,}048{,}576 = 20$	$\lfloor \log_2 1{,}048{,}576 \rfloor + 1 = 21$

Riepilogo

Punti chiave sui numeri naturali:

- La sequenza $(d_{k-1} \dots d_1 d_0)_b$ rappresenta il numero naturale $\sum_{i=0}^{k-1} d_i \, b^i$
- Con k cifre in base b si rappresentano esattamente i b^k numeri naturali dell'intervallo $[0, b^k - 1]$
- Un numero naturale $n \geq 1$ richiede almeno $\lfloor \log_b n \rfloor + 1$ cifre in base b

Rappresentazione dei numeri interi

Introduzione: Rappresentazione dei numeri interi

- I numeri interi comprendono sia valori positivi sia negativi, più lo zero
- Con k bit abbiamo ancora soltanto 2^k sequenze diverse, ma ora dobbiamo usarle anche per rappresentare il **segno**
- Due approcci:
  - il **modulo e segno**, il più naturale
  - il **complemento a due**, lo standard nei calcolatori moderni per i suoi vantaggi aritmetici
- In questa sezione la base è sempre $b = 2$

Rappresentazione in modulo e segno

Definizione 4: rappresentazione in modulo e segno

*Nella **rappresentazione in modulo e segno**, un numero intero n è rappresentato usando esattamente $k \geq 2$ bit, dove:*

- *il bit più significativo rappresenta il **segno**: 0 se $n \geq 0$ e 1 se $n < 0$;*
- *i restanti $k - 1$ bit rappresentano il **modulo** $|n|$ (valore assoluto) in base 2.*

Osservazione 3

Dati k bit, il ***più grande numero intero*** *rappresentabile in modulo e segno è:*

$$2^{k-1} - 1$$

e il relativo ***intervallo di rappresentazione*** *per i numeri interi è:*

$$\left[-(2^{k-1} - 1),\ 2^{k-1} - 1 \right]$$

Dimostrazione

Il modulo è un numero naturale scritto con $k-1$ bit.

Per l'Osservazione 1 con $b=2$ e $k-1$ cifre, il modulo più grande è formato dalla cifra 1 ripetuta $k-1$ volte:

$$\sum_{i=0}^{k-2} 1 \cdot 2^i = \frac{2^{k-1}-1}{2-1} = 2^{k-1}-1$$

- Con bit di segno 0 si ottiene il più grande intero $2^{k-1}-1$
- Con bit di segno 1 si ottiene il più piccolo intero $-(2^{k-1}-1)$
- Ogni intero intermedio è rappresentato, poiché il suo valore assoluto è al più $2^{k-1}-1$

□

Esempio: modulo e segno con 4 bit (1/2)

Con $k = 4$ bit, il primo bit è il segno e gli ultimi tre bit sono il modulo:

Binario	k	Segno	Decimale
$(0000)_2$	4	0 (non negativo)	$+(0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0) = +0$
$(0001)_2$	4	0 (non negativo)	$+(0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0) = 1$
$(0010)_2$	4	0 (non negativo)	$+(0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = 2$
$(0011)_2$	4	0 (non negativo)	$+(0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) = 3$
$(0100)_2$	4	0 (non negativo)	$+(1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0) = 4$
$(0101)_2$	4	0 (non negativo)	$+(1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0) = 5$
$(0110)_2$	4	0 (non negativo)	$+(1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = 6$
$(0111)_2$	4	0 (non negativo)	$+(1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) = 7$

Esempio: modulo e segno con 4 bit (2/2)

Binario	k	Segno	Decimale
$(1000)_2$	4	1 (negativo)	$-(0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0) = -0$
$(1001)_2$	4	1 (negativo)	$-(0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0) = -1$
$(1010)_2$	4	1 (negativo)	$-(0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = -2$
$(1011)_2$	4	1 (negativo)	$-(0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) = -3$
$(1100)_2$	4	1 (negativo)	$-(1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0) = -4$
$(1101)_2$	4	1 (negativo)	$-(1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0) = -5$
$(1110)_2$	4	1 (negativo)	$-(1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = -6$
$(1111)_2$	4	1 (negativo)	$-(1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) = -7$

L'intervallo di rappresentazione è $[-7, 7]$, come afferma l'Osservazione 3 con $2^{4-1} - 1 = 7$.

Punti deboli del modulo e segno (1/2)

La tabella dell'esempio mostra i due punti deboli del modulo e segno.

1. **Lo zero ha due rappresentazioni**
   - $(0000)_2 = +0$ e $(1000)_2 = -0$
   - Quindi $2^k - 1$ interi diversi sono rappresentati con 2^k sequenze

Punti deboli del modulo e segno (2/2)

2. La somma non è la somma dei numeri naturali

- La somma di due interi non può essere eseguita come somma delle due sequenze viste come numeri naturali
- Ad esempio, $3 + (-1) = 2$, ma sommando $(0011)_2$ e $(1001)_2$ come numeri naturali si ottiene

$$3 + 9 = 12 = (1100)_2$$

  che in modulo e segno rappresenta -4
- I bit di segno vanno trattati separatamente, e questo complica le operazioni aritmetiche

Per questi motivi il modulo e segno non è la rappresentazione standard degli interi nei calcolatori.

Rappresentazione in complemento a due

Il complemento a due risolve entrambi i problemi:

lo zero ha una rappresentazione unica

e la somma funziona esattamente come per i numeri naturali.

Definizione 5: rappresentazione in complemento a due

*Nella **rappresentazione in complemento a due** (base 2), un intero n è rappresentato usando esattamente k bit nel modo seguente:*

- *se $n \geq 0$, si usa la rappresentazione binaria standard di n con k bit;*
- *se $n < 0$, la rappresentazione di n è la rappresentazione binaria del numero naturale $2^k + n$ con k bit.*

Il bit di segno nel complemento a due

Nel complemento a due, il bit più significativo indica il segno:

- 0: numero non negativo;
- 1: numero negativo.

Per leggere una sequenza il cui bit più significativo è 1, si calcola il suo valore r come numero naturale: l'intero rappresentato è $r - 2^k$.

Proprietà del complemento a due

- Lo zero ha una rappresentazione unica: $(00\ldots0)_2$
- Somma e sottrazione si eseguono come per i numeri naturali, modulo 2^k
- È la rappresentazione standard usata nei calcolatori moderni

Esempio: complemento a due con 4 bit (1/2)

Con $k = 4$ bit, le sequenze con bit più significativo 0 rappresentano $0, 1, \ldots, 7$:

Binario	k	Segno	Decimale
$(0000)_2$	4	non negativo	0
$(0001)_2$	4	non negativo	1
$(0010)_2$	4	non negativo	2
$(0011)_2$	4	non negativo	3
$(0100)_2$	4	non negativo	4
$(0101)_2$	4	non negativo	5
$(0110)_2$	4	non negativo	6
$(0111)_2$	4	non negativo	7

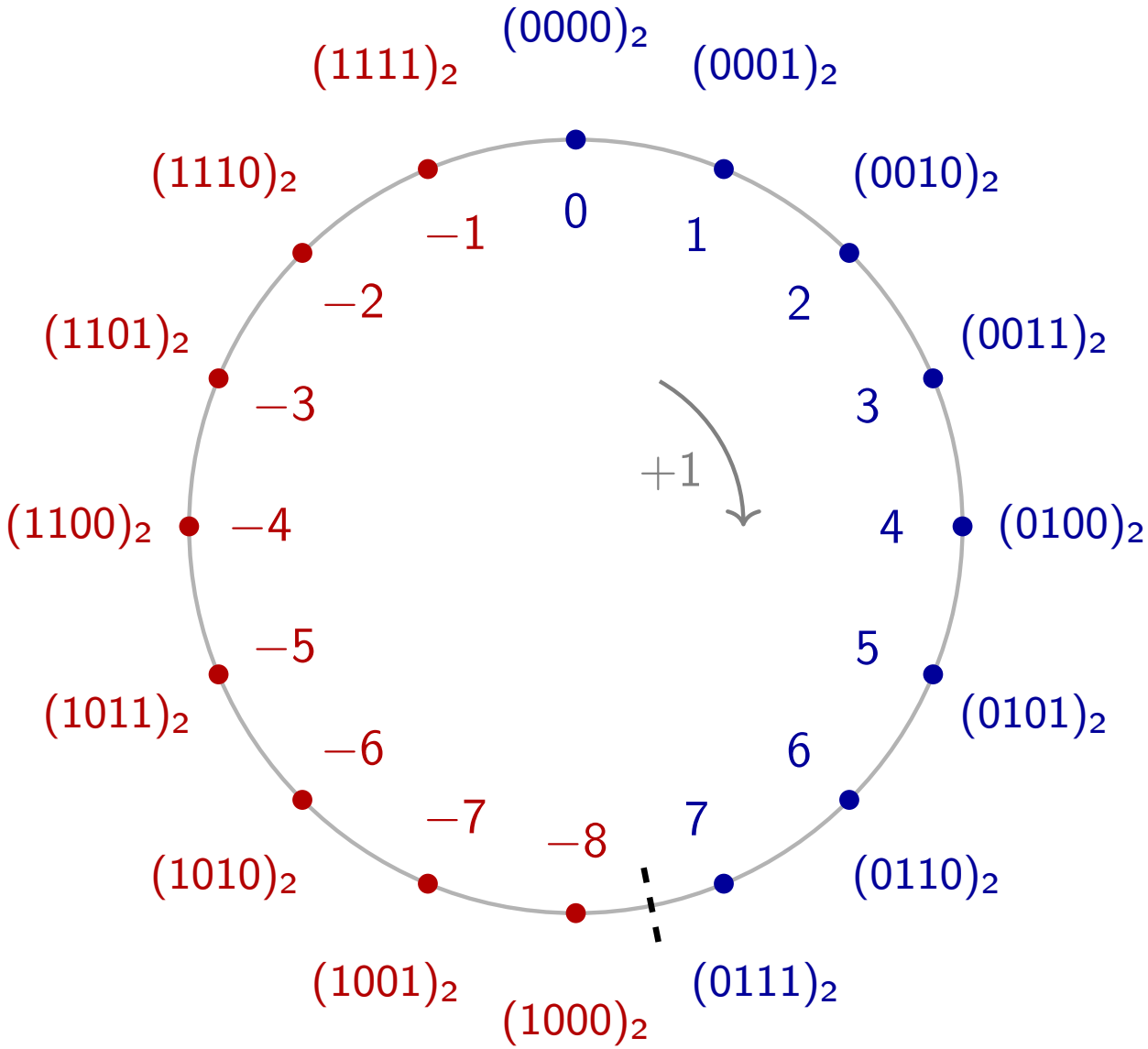
Esempio: complemento a due con 4 bit (2/2)

Le sequenze con bit più significativo 1 rappresentano il loro valore naturale meno $2^4 = 16$:

Binario	k	Segno	Decimale
$(1000)_2$	4	negativo	$8 - 16 = -8$
$(1001)_2$	4	negativo	$9 - 16 = -7$
$(1010)_2$	4	negativo	$10 - 16 = -6$
$(1011)_2$	4	negativo	$11 - 16 = -5$
$(1100)_2$	4	negativo	$12 - 16 = -4$
$(1101)_2$	4	negativo	$13 - 16 = -3$
$(1110)_2$	4	negativo	$14 - 16 = -2$
$(1111)_2$	4	negativo	$15 - 16 = -1$

La ruota del complemento a due

- La figura seguente dispone le 16 sequenze dell'esempio su una circonferenza
- Avanzare di un passo in senso orario aggiunge 1 al valore naturale della sequenza, modulo $2^4 = 16$
- Interi non negativi in blu (bit più significativo 0), interi negativi in rosso (bit più significativo 1)
- Gli interi rappresentati crescono da -8 a 7: il segno tratteggiato tra $(0111)_2 = 7$ e $(1000)_2 = -8$ è l'unico punto in cui l'intero rappresentato fa un salto


$(0000)_2$
$(0001)_2$
$(0010)_2$
$(0011)_2$
$(0100)_2$
$(0101)_2$
$(0110)_2$
$(0111)_2$
$(1000)_2$
$(1001)_2$
$(1010)_2$
$(1011)_2$
$(1100)_2$
$(1101)_2$
$(1110)_2$
$(1111)_2$
0
1
2
3
4
5
6
7
−8
−7
−6
−5
−4
−3
−2
−1
+1

Calcolare $2^k + n$ con una sottrazione non è l'unico modo per ottenere la rappresentazione di un intero negativo.

La seguente osservazione fornisce una procedura che lavora direttamente sui bit.

Osservazione 4

Sia n un intero con $-2^{k-1} \leq n < 0$. La rappresentazione in complemento a due di n con k bit si ottiene dalla rappresentazione binaria di $|n|$ con k bit in due passi:

1. *si invertono tutti i bit (ogni 0 diventa 1 e ogni 1 diventa 0);*
2. *si aggiunge 1 al risultato, come numero naturale.*

Dimostrazione

Sia $(d_{k-1} \dots d_1 d_0)_2$ la rappresentazione binaria di $|n|$ con k bit.

Invertendo i bit si ottengono le cifre $1 - d_i$, quindi il numero naturale:

$$\sum_{i=0}^{k-1} (1 - d_i)\, 2^i = \sum_{i=0}^{k-1} 2^i - \sum_{i=0}^{k-1} d_i\, 2^i$$

$$= (2^k - 1) - |n|$$

- Aggiungendo 1 si ottiene $2^k - |n| = 2^k + n$, che è la rappresentazione in complemento a due di n per la Definizione 5
- Poiché $1 \leq |n| \leq 2^{k-1}$, si ha $2^{k-1} \leq 2^k + n \leq 2^k - 1$, quindi il risultato ha ancora k bit

□

Esempio: complemento a due di -5 con 8 bit

Calcoliamo la rappresentazione di $n = -5$ con $k = 8$ bit in entrambi i modi.

- Passo 1: rappresentazione binaria di $|n| = 5$ con 8 bit: $(00000101)_2$
- Passo 2: si invertono tutti i bit: $(11111010)_2$, il cui valore naturale è $250 = (2^8 - 1) - 5$
- Passo 3: si aggiunge 1: $(11111010)_2 + 1 = (11111011)_2$
- Verifica con la Definizione 5:

$$\begin{aligned} 2^8 + (-5) &= 256 - 5 = 251 \\ &= 128 + 64 + 32 + 16 + 8 + 2 + 1 = (11111011)_2 \end{aligned}$$

Intervallo di rappresentazione

Quali interi si possono rappresentare
con k bit in complemento a due?

Osservazione 5

Dati k bit, il più grande intero rappresentabile in complemento a due è:

$$2^{k-1} - 1$$

il più piccolo intero rappresentabile è:

$$-2^{k-1}$$

e quindi l'intervallo di rappresentazione è:

$$\left[-2^{k-1}, 2^{k-1} - 1 \right]$$

Dimostrazione

Con k bit ci sono 2^k possibili stringhe binarie, i cui valori naturali sono $0, 1, \ldots, 2^k - 1$.

- Le stringhe il cui bit più significativo è 0 hanno valori naturali $0, 1, \ldots, 2^{k-1} - 1$ e rappresentano gli interi non negativi:

$$0, 1, 2, \ldots, 2^{k-1} - 1$$

- Le stringhe il cui bit più significativo è 1 hanno valori naturali $r = 2^{k-1}, \ldots, 2^k - 1$ e rappresentano gli interi $r - 2^k$, cioè gli interi negativi:

$$-2^{k-1}, -2^{k-1} + 1, \ldots, -1$$

Quindi il più grande intero rappresentabile è $2^{k-1} - 1$, il più piccolo è -2^{k-1}, e l'intervallo è $[-2^{k-1}, 2^{k-1} - 1]$. □

Confronto con il modulo e segno

- Rispetto al modulo e segno, l'intervallo contiene un intero in più, -2^{k-1}
- La sequenza $(10\ldots0)_2$, che era -0 in modulo e segno, ora rappresenta -2^{k-1}
- Tutte le 2^k sequenze rappresentano interi diversi

Esempio: intervalli di rappresentazione in complemento a due

k	-2^{k-1}	$2^{k-1}-1$	intervallo di rappresentazione
1	-1	0	$[-1, 0]$
2	-2	1	$[-2, 1]$
3	-4	3	$[-4, 3]$
4	-8	7	$[-8, 7]$
8	-128	127	$[-128, 127]$
16	$-32{,}768$	32,767	$[-32{,}768,\ 32{,}767]$
32	$-2{,}147{,}483{,}648$	2,147,483,647	$[-2{,}147{,}483{,}648,\ 2{,}147{,}483{,}647]$[1]
64	-2^{63}	$2^{63}-1$	$[-2^{63},\ 2^{63}-1]$[2]

[1] Intervallo di rappresentazione di `int` in C e C++ (4 byte).
[2] Intervallo di rappresentazione di `long long int` in C e C++ (8 byte); $2^{63} = 9{,}223{,}372{,}036{,}854{,}775{,}808$.

Numero minimo di bit

Come per i numeri naturali, leggiamo ora l'intervallo nel verso opposto:

quanti bit servono
per rappresentare un dato intero n?

Osservazione 6

Il numero minimo di bit necessari per rappresentare un intero n in complemento a due è il più piccolo intero $k \geq 1$ tale che

$$-2^{k-1} \leq n \leq 2^{k-1} - 1.$$

Dimostrazione

Per l'Osservazione 5, un intero rappresentato con k bit in complemento a due appartiene all'intervallo

$$[-2^{k-1}, 2^{k-1} - 1],$$

e ogni intero di questo intervallo può essere rappresentato con k bit.

Perciò il numero minimo di bit necessari per rappresentare n è esattamente il più piccolo k per cui

$$-2^{k-1} \leq n \leq 2^{k-1} - 1.$$

Come per i numeri naturali, questa condizione si può trasformare
in una formula esplicita con i logaritmi.

Corollario 1

Il numero minimo di bit necessari per rappresentare un intero n in complemento a due è:

$$k = \begin{cases} 1 & \text{se } n = 0 \text{ o } n = -1 \\ \lfloor \log_2 n \rfloor + 2 & \text{se } n > 0 \\ \lceil \log_2 |n| \rceil + 1 & \text{se } n < 0 \end{cases}$$

Dimostrazione

Con $k = 1$ l'intervallo è $[-1, 0]$, e questo dà il primo caso.

- Se $n > 0$, la condizione dell'Osservazione 6 si riduce a

$$n \leq 2^{k-1} - 1 \iff n < 2^{k-1} \iff \log_2 n < k - 1$$

Il più piccolo intero $k - 1$ strettamente maggiore di $\log_2 n$ è $\lfloor \log_2 n \rfloor + 1$, quindi $k = \lfloor \log_2 n \rfloor + 2$.

- Se $n < 0$, la condizione si riduce a

$$-2^{k-1} \leq n \iff |n| \leq 2^{k-1} \iff \log_2 |n| \leq k - 1$$

Il più piccolo intero $k - 1$ maggiore o uguale a $\log_2 |n|$ è $\lceil \log_2 |n| \rceil$, quindi $k = \lceil \log_2 |n| \rceil + 1$ (per $n = -1$ si ottiene $k = 1$, in accordo con il primo caso).



Esempio: numero minimo di bit in complemento a due

L'ultima colonna mostra la rappresentazione di n con il numero minimo di bit.

n	numero minimo di bit	rappresentazione
-1	1	1
0	1	0
1	2	01
-2	2	10
2	3	010
-4	3	100
4	4	0100
-8	4	1000
8	5	01000
-16	5	10000

Ad esempio, per $n = 8$ il Corollario 1 dà $\lfloor \log_2 8 \rfloor + 2 = 3 + 2 = 5$, e per $n = -16$ dà $\lceil \log_2 16 \rceil + 1 = 4 + 1 = 5$.

Somma e overflow

- Il principale vantaggio del complemento a due: gli interi si sommano **esattamente come i numeri naturali**
- Si sommano le due sequenze di k bit colonna per colonna, da destra a sinistra, con i riporti
- Si scarta il riporto che esce dalla posizione più significativa

Osservazione 7

Siano n_1 e n_2 due interi rappresentati con k bit in complemento a due. Si sommino le loro rappresentazioni come numeri naturali e si tengano soltanto i k bit meno significativi della somma.

- *Se $n_1 + n_2 \in [-2^{k-1}, 2^{k-1} - 1]$, il risultato è la rappresentazione in complemento a due di $n_1 + n_2$.*
- *Altrimenti, la somma non può essere rappresentata con k bit e si ha un **overflow**. Un overflow si verifica se e solo se n_1 e n_2 hanno lo stesso segno e il risultato ha il bit di segno opposto.*

Dimostrazione (1/3)

Siano r_1 e r_2 i valori naturali delle due rappresentazioni.

- Per la Definizione 5, $r_j = n_j$ oppure $r_j = 2^k + n_j$
- Quindi $r_1 + r_2$ differisce da $n_1 + n_2$ di 0, 2^k o 2^{k+1}
- Tenere i k bit meno significativi significa sottrarre 2^k quando $r_1 + r_2 \geq 2^k$ (il riporto scartato)

Quindi il risultato r soddisfa

$$r \in [0, 2^k - 1] \quad \text{e} \quad r - (n_1 + n_2) \text{ è un multiplo di } 2^k$$

Dimostrazione (2/3)

La somma appartiene all'intervallo.

Se $n_1 + n_2 \in [-2^{k-1}, 2^{k-1} - 1]$:

- anche la sua rappresentazione $\bar{r}$ appartiene a $[0, 2^k - 1]$
- $\bar{r} - (n_1 + n_2)$ vale 0 oppure 2^k
- allora $r - \bar{r}$ è un multiplo di 2^k con $|r - \bar{r}| \leq 2^k - 1$, quindi $r = \bar{r}$

Segni diversi.

Se n_1 e n_2 hanno segni diversi, $n_1 + n_2$ sta tra n_1 e n_2, che appartengono all'intervallo: nessun overflow.

Dimostrazione (3/3)

Entrambi non negativi.

- $n_1 + n_2 \in [0, 2^k - 2]$, nessun riporto viene scartato e $r = n_1 + n_2$
- Si ha overflow se e solo se $n_1 + n_2 \geq 2^{k-1}$, cioè se e solo se il bit di segno di r è 1

Entrambi negativi.

- $r_1 + r_2 = 2^{k+1} + n_1 + n_2 \geq 2^k$, quindi $r = 2^k + n_1 + n_2$
- Si ha overflow se e solo se $n_1 + n_2 < -2^{k-1}$, cioè se e solo se $r < 2^{k-1}$, cioè il bit di segno di r è 0



Come leggere le somme in colonna

- Negli esempi, la prima riga di ogni somma contiene i **riporti**
- Il riporto scritto sopra una colonna è quello che arriva dalla colonna alla sua destra
- La colonna più a sinistra riceve il riporto uscente dalla posizione più significativa, che viene scartato

Esempio: somma senza overflow

Calcoliamo $3 + (-1)$ con $k = 4$ bit, intervallo $[-8, 7]$.

- Passo 1: rappresentazioni: $3 = (0011)_2$ e $-1 \to 2^4 - 1 = 15 = (1111)_2$
- Passo 2: somma in colonna:

riporto	1	1	1	1	
		0	0	1	1
		1	1	1	1
somma	1	0	0	1	0

- Passo 3: si scarta il riporto uscente: il risultato è $(0010)_2 = 2 = 3 + (-1)$

Esempio: somma con overflow

Calcoliamo $6 + 3$ con $k = 4$ bit, intervallo $[-8, 7]$. Poiché $6 + 3 = 9 > 7$, ci aspettiamo un overflow.

- Passo 1: rappresentazioni: $6 = (0110)_2$ e $3 = (0011)_2$
- Passo 2: somma in colonna:

riporto	0	1	1	0	
		0	1	1	0
		0	0	1	1
somma	0	1	0	0	1

- Passo 3: il risultato $(1001)_2$ ha bit di segno 1 e rappresenta $9 - 16 = -7$, benché entrambi gli operandi siano positivi: si è verificato un overflow

Riepilogo

Punti chiave sui numeri interi:

- Modulo e segno con k bit: intervallo $[-(2^{k-1}-1), 2^{k-1}-1]$, due rappresentazioni dello zero, la somma richiede un trattamento speciale del segno
- Complemento a due con k bit: un n negativo è rappresentato da $2^k + n$ (equivalentemente: si invertono i bit di $|n|$ e si aggiunge 1); intervallo $[-2^{k-1}, 2^{k-1}-1]$; zero unico
- Nel complemento a due, la somma è la somma di numeri naturali modulo 2^k; si ha overflow solo quando due operandi con lo stesso segno danno un risultato con il segno opposto

Rappresentazione dei numeri reali

Introduzione: Rappresentazione dei numeri reali

- I numeri naturali e interi sono rappresentati esattamente, purché appartengano all'intervallo di rappresentazione
- I numeri reali sono diversi: tra due numeri reali qualsiasi ce ne sono infiniti altri
- La maggior parte dei numeri reali (ad esempio $\frac{1}{3}$ o $\sqrt{2}$) ha infinite cifre in qualunque base
- Con un numero finito di bit, un numero reale può essere rappresentato, in generale, solo in modo **approssimato**
- Studiamo la rappresentazione in virgola mobile usata dai calcolatori

Cifre dopo la virgola

- I sistemi di numerazione posizionali si estendono alle cifre dopo la virgola
- Le cifre a destra della virgola si moltiplicano per le potenze negative della base

Definizione 6: sequenza di cifre con parte frazionaria

Una sequenza di k cifre prima della virgola e h cifre dopo la virgola in base b si indica con:

$$\left(d_{k-1} \ldots d_1\, d_0 \,.\, d_{-1}\, d_{-2} \ldots d_{-h}\right)_b$$

e rappresenta il numero:

$$\sum_{i=-h}^{k-1} d_i \; b^i$$

Esempio: numeri binari con parte frazionaria

- $(101.11)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2}$

$$= 4 + 1 + 0.5 + 0.25 = 5.75$$

- $(0.011)_2 = 0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.25 + 0.125 = 0.375$
- Il numero decimale 0.1 non ha una rappresentazione binaria finita: le sue cifre binarie dopo la virgola si ripetono all'infinito,

$$0.1 = (0.0\,0011\,0011\,0011\ldots)_2$$

Virgola fissa e virgola mobile

- Scrivere un numero con un numero fissato di cifre prima e dopo la virgola (**virgola fissa**) spreca cifre:
  - i numeri molto grandi richiedono molte cifre prima della virgola
  - quelli molto piccoli molte cifre dopo

- La rappresentazione in **virgola mobile** lascia "fluttuare" la virgola, come nella notazione scientifica

$$6.02 \times 10^{23}$$

Definizione 7: notazione scientifica binaria normalizzata

Ogni numero reale $x \neq 0$ *si può scrivere nella* ***notazione scientifica binaria normalizzata****:*

$$x = (-1)^s \cdot (1.f_1 f_2 f_3 \dots)_2 \cdot 2^e$$

dove $s \in \{0, 1\}$ *è il* ***segno****, la sequenza* $(1.f_1 f_2 f_3 \dots)_2$ *è la* ***mantissa*** *(o significand), i cui bit* $f_1, f_2, f_3, \dots$ *dopo la virgola formano la* ***parte frazionaria****, e l'intero* e *è l'****esponente****.*

La mantissa

- La mantissa è sempre compresa tra 1 (incluso) e 2 (escluso)
- La sua prima cifra è sempre 1, quindi non serve memorizzarla
- Ad esempio:

$$5.75 = (101.11)_2 = (1.0111)_2 \cdot 2^2$$

la virgola si sposta di due posizioni a sinistra e l'esponente è 2

Lo standard IEEE 754

- I calcolatori memorizzano i numeri in virgola mobile secondo lo standard **IEEE 754**
  - IEEE: Institute of Electrical and Electronics Engineers
- Ne descriviamo i due formati più comuni, concentrandoci su quello a 32 bit

Definizione 8: IEEE 754 singola precisione

*Nel formato **IEEE 754 a singola precisione**, un numero reale è memorizzato con 32 bit divisi in tre campi:*

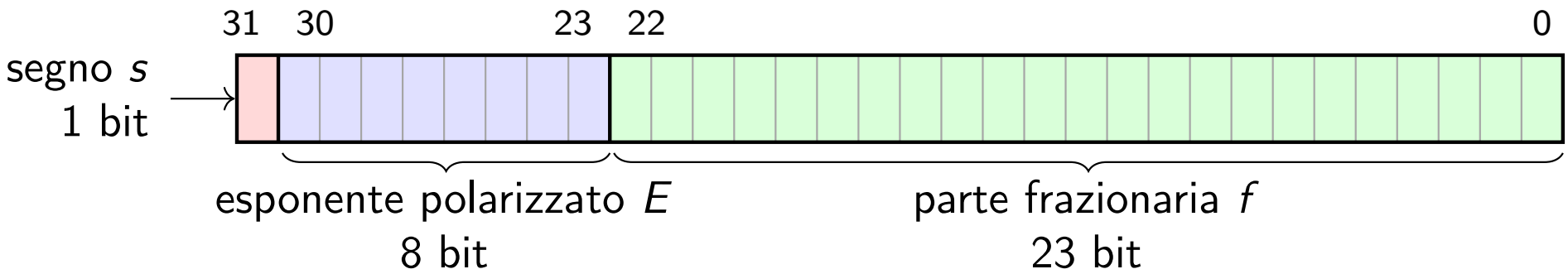
- *il **segno** s: 1 bit;*
- *l'**esponente polarizzato** E: 8 bit, che memorizzano il numero naturale $E = e + 127$, dove e è l'esponente e 127 è la **polarizzazione** (bias);*
- *la **parte frazionaria** f: 23 bit, che memorizzano i bit $f_1 f_2 \dots f_{23}$ della mantissa dopo la virgola.*

*Per $1 \leq E \leq 254$ (**numeri normalizzati**), il numero rappresentato è:*

$$x = (-1)^s \cdot (1.f_1 f_2 \dots f_{23})_2 \cdot 2^{E-127}$$

I 32 bit della singola precisione

I 32 bit del formato IEEE 754 a singola precisione, numerati da 31 (più significativo) a 0 (meno significativo):



Il campo dell'esponente

- L'esponente è memorizzato con una polarizzazione anziché in complemento a due, in modo che E sia un numero naturale tra 0 e 255
- I due valori estremi di E sono riservati:
  - $E = 0$ si usa per lo zero (tutti i bit della parte frazionaria uguali a 0; ci sono due zeri, $+0$ e -0, come nel modulo e segno) e per i numeri molto piccoli detti *subnormali*
  - $E = 255$ si usa per $+\infty$ e $-\infty$ (parte frazionaria uguale a 0, ad esempio il risultato di $1/0$) e per *NaN*, "Not a Number" (parte frazionaria diversa da 0, ad esempio il risultato di $0/0$)

Singola e doppia precisione

- Il formato a **doppia precisione** ha la stessa struttura con 64 bit: 1 bit di segno, 11 bit di esponente con polarizzazione 1023 e 52 bit di parte frazionaria

Formato	bit totali	segno	esponente	parte frazionaria	polarizzazione
singola precisione (C++ `float`)	32	1	8	23	127
doppia precisione (C++ `double`)	64	1	11	52	1023

Esempio: da un numero reale alla sua rappresentazione IEEE 754 a singola precisione

Rappresentiamo $x = 5.75$ in singola precisione.

- Passo 1 (binario): $5.75 = 4 + 1 + 0.5 + 0.25 = (101.11)_2$
- Passo 2 (normalizzazione): $(101.11)_2 = (1.0111)_2 \cdot 2^2$, quindi $s = 0$, $e = 2$ e la parte frazionaria inizia con 0111
- Passo 3 (esponente polarizzato): $E = e + 127 = 129 = 128 + 1 = (10000001)_2$
- Passo 4 (parte frazionaria): i bit dopo la virgola, completati con zeri fino a 23 bit: 01110000000000000000000

I 32 bit sono:

$$\underbrace{0}_{s} \quad \underbrace{10000001}_{E} \quad \underbrace{01110000000000000000000}_{f}$$

Esempio: da una rappresentazione IEEE 754 a singola precisione a un numero reale

Leggiamo i 32 bit:

$$\underbrace{1}_{s} \quad \underbrace{10000010}_{E} \quad \underbrace{01000000000000000000000}_{f}$$

- Passo 1 (segno): $s = 1$, il numero è negativo
- Passo 2 (esponente): $E = (10000010)_2 = 128 + 2 = 130$, quindi $e = E - 127 = 3$
- Passo 3 (mantissa): $(1.01)_2 = 1 + 0.25 = 1.25$
- Passo 4 (valore): $x = (-1)^1 \cdot 1.25 \cdot 2^3 = -10$

Arrotondamento

Quando le cifre binarie di un numero
non entrano nei 23 bit della parte frazionaria,

il numero viene **arrotondato**
al più vicino numero rappresentabile.

Esempio: il numero 0.1 non è rappresentabile esattamente (1/2)

- Dall'esempio sui numeri binari con parte frazionaria:

$$0.1 = (0.0\,0011\,0011\ldots)_2 = (1.1001\,1001\,1001\ldots)_2 \cdot 2^{-4}$$

  la cui parte frazionaria non termina mai

- In singola precisione la parte frazionaria viene arrotondata a 23 bit, e i bit memorizzati sono:

$$\underbrace{0}_{s} \quad \underbrace{01111011}_{E=123} \quad \underbrace{10011001100110011001101}_{f}$$

Esempio: il numero 0.1 non è rappresentabile esattamente (2/2)

- Il numero effettivamente memorizzato è 0.1000000014901161193847656250, non 0.1
- In doppia precisione l'errore è più piccolo ma comunque presente: il valore memorizzato è

$$0.10000000000000000055511151231\ldots$$

- Per questo motivo il calcolo $0.1 + 0.2$ in doppia precisione dà

$$0.30000000000000004$$

e non il valore in doppia precisione di 0.3

Campo di variazione e precisione

- Il numero finito di bit dell'**esponente** limita il **campo di variazione** (*range*) dei numeri rappresentabili
- Il numero finito di bit della **parte frazionaria** ne limita la **precisione**

Osservazione 8

Nello standard IEEE 754 a singola precisione:

- *il più grande numero finito è* $(2 - 2^{-23}) \cdot 2^{127} \approx 3.4028235 \times 10^{38}$;
- *il più piccolo numero normalizzato positivo è* $2^{-126} \approx 1.1754944 \times 10^{-38}$;
- *la distanza tra* 1 *e il successivo numero rappresentabile è* $2^{-23} \approx 1.1920929 \times 10^{-7}$ *(detta* ***epsilon macchina****).*

Dimostrazione (1/2)

Il più grande numero finito.

Ha $s = 0$, il più grande esponente polarizzato normalizzato $E = 254$, cioè $e = 127$, e tutti i 23 bit della parte frazionaria uguali a 1. La sua mantissa è:

$$(1.\underbrace{11\ldots1}_{23})_2 = \sum_{i=0}^{23} 2^{-i} = 2 - 2^{-23}$$

quindi il numero è $(2 - 2^{-23}) \cdot 2^{127}$.

Dimostrazione (2/2)

Il più piccolo numero normalizzato positivo.

Ha $E = 1$, cioè $e = -126$, e tutti i bit della parte frazionaria uguali a 0, quindi vale $1 \cdot 2^{-126}$.

Epsilon macchina.

Si ha $1 = (1.00\ldots0)_2 \cdot 2^0$. Il successivo numero rappresentabile si ottiene ponendo a 1 l'ultimo bit della parte frazionaria, che ha peso 2^{-23} quando $e = 0$: è $1 + 2^{-23}$. □

Precisione relativa

- L'epsilon macchina misura la **precisione relativa** del formato:
  - la singola precisione corrisponde a circa 7 cifre decimali significative
  - la doppia precisione (epsilon macchina $2^{-52} \approx 2.2 \times 10^{-16}$) a circa 16
- Di conseguenza, due espressioni matematicamente uguali possono dare risultati diversi in virgola mobile
- I numeri in virgola mobile non vanno confrontati con l'operatore di uguaglianza senza cautela

Riepilogo

Punti chiave sui numeri reali:

- Un numero in virgola mobile è memorizzato come segno, esponente e parte frazionaria: $x = (-1)^s \cdot (1.f)_2 \cdot 2^{E-\text{bias}}$
- IEEE 754 singola precisione: $1 + 8 + 23$ bit, polarizzazione 127; doppia precisione: $1 + 11 + 52$ bit, polarizzazione 1023
- I bit dell'esponente determinano il campo di variazione, i bit della parte frazionaria la precisione; la maggior parte dei numeri reali (ad esempio 0.1) è memorizzata con un errore di arrotondamento

In sintesi

Introduzione: In sintesi

- Raccogliamo le tre rappresentazioni del capitolo
- Per ciascuna: che cosa si rappresenta con un dato numero di cifre, e le proprietà principali

Riepilogo (1/2)

Numeri naturali (k cifre in base b):

- valore $\sum_{i=0}^{k-1} d_i \, b^i$, intervallo $[0, b^k - 1]$
- almeno $\lfloor \log_b n \rfloor + 1$ cifre per $n \geq 1$

Numeri interi (k bit):

- modulo e segno: intervallo $[-(2^{k-1} - 1), 2^{k-1} - 1]$, due zeri
- complemento a due: $n < 0$ memorizzato come $2^k + n$, intervallo $[-2^{k-1}, 2^{k-1} - 1]$, zero unico
- somma in complemento a due: somma dei naturali modulo 2^k, overflow solo per operandi con lo stesso segno

Riepilogo (2/2)

Numeri reali (virgola mobile):

- $x = (-1)^s \cdot (1.f)_2 \cdot 2^{E-\text{bias}}$
- IEEE 754: singola precisione $1 + 8 + 23$ bit, polarizzazione 127; doppia precisione $1 + 11 + 52$ bit, polarizzazione 1023
- l'esponente dà il campo di variazione, la parte frazionaria la precisione
- la maggior parte dei numeri reali, come 0.1, è memorizzata con un errore di arrotondamento